

Program Optimization

Introduction to Computer Systems
10th Lecture

Instructors:

Yili Gong

Today

- **Overview**
- **Generally Useful Optimizations**
 - Code motion/precomputation
 - Strength reduction
 - Sharing of common subexpressions
 - Removing unnecessary procedure calls
- **Optimization Blockers**
 - Procedure calls
 - Memory aliasing
- **Exploiting Instruction-Level Parallelism**
- **Dealing with Conditionals**

Performance Realities

- **There's more to performance than asymptotic complexity**
- **Constant factors matter too!**
 - Easily see 10:1 performance range depending on how code is written
 - Must optimize at multiple levels:
 - algorithm, data representations, procedures, and loops
- **Must understand system to optimize performance**
 - How programs are compiled and executed
 - How modern processors + memory systems operate
 - How to measure program performance and identify bottlenecks
 - How to improve performance without destroying code modularity and generality

Optimizing Compilers

- **Provide efficient mapping of program to machine**
 - register allocation
 - code selection and ordering (scheduling)
 - dead code elimination
 - eliminating minor inefficiencies
- **Don't (usually) improve asymptotic efficiency**
 - up to programmer to select best overall algorithm
 - big-O savings are (often) more important than constant factors
 - but constant factors also matter
- **Have difficulty overcoming “optimization blockers”**
 - potential memory aliasing
 - potential procedure side-effects

Limitations of Optimizing Compilers

- **Operate under fundamental constraint**
 - Must not cause any change in program behavior
 - Except, possibly when program making use of nonstandard language features
 - Often prevents it from making optimizations that would only affect behavior under pathological conditions.
- **Behavior that may be obvious to the programmer can be obfuscated by languages and coding styles**
 - e.g., Data ranges may be more limited than variable types suggest
- **Most analysis is performed only within procedures**
 - Whole-program analysis is too expensive in most cases
 - Newer versions of GCC do inter-procedural analysis within individual files
 - But, not between code in different files
- **Most analysis is based only on *static* information**
 - Compiler has difficulty anticipating run-time inputs
- **When in doubt, the compiler must be conservative**

```
1 void twiddle1(long *xp, long *yp)
2 {
3     *xp += *yp;
4     *xp += *yp;
5 }
6
7 void twiddle2(long *xp, long *yp)
8 {
9     *xp += 2* *yp;
10 }
```

Generally Useful Optimizations

- Optimizations that you or the compiler should do regardless of processor / compiler
- Code Motion
 - Reduce frequency with which computation performed
 - If it will always produce same result
 - Especially moving code out of loop

```
void set_row(double *a, double *b,  
            long i, long n)  
{  
    long j;  
    for (j = 0; j < n; j++)  
        a[n*i+j] = b[j];  
}
```



```
long j;  
int ni = n*i;  
for (j = 0; j < n; j++)  
    a[ni+j] = b[j];
```

Compiler-Generated Code Motion (-O1)

```
void set_row(double *a, double *b,  
            long i, long n)  
{  
    long j;  
    for (j = 0; j < n; j++)  
        a[n*i+j] = b[j];  
}
```

```
long j;  
long ni = n*i;  
double *rowp = a+ni;  
for (j = 0; j < n; j++)  
    *rowp++ = b[j];
```

```
set_row:  
    testq    %rcx, %rcx          # Test n  
    jle      .L1                 # If 0, goto done  
    imulq    %rcx, %rdx          # ni = n*i  
    leaq     (%rdi,%rdx,8), %rdx  # rowp = A + ni*8  
    movl     $0, %eax            # j = 0  
.L3:                                # loop:  
    movsd    (%rsi,%rax,8), %xmm0 # t = b[j]  
    movsd    %xmm0, (%rdx,%rax,8) # M[A+ni*8 + j*8] = t  
    addq     $1, %rax            # j++  
    cmpq     %rcx, %rax          # j:n  
    jne      .L3                 # if !=, goto loop  
.L1:                                # done:  
    rep ; ret
```


Reduction in Strength

- Replace costly operation with simpler one
- Shift, add instead of multiply or divide
 - $16 * x \rightarrow x \ll 4$
 - Utility machine dependent
 - Depends on cost of multiply or divide instruction
 - On Intel Nehalem, integer multiply requires 3 CPU cycles
- Recognize sequence of products

```
for (i = 0; i < n; i++) {  
    int ni = n*i;  
    for (j = 0; j < n; j++)  
        a[ni + j] = b[j];  
}
```



```
int ni = 0;  
for (i = 0; i < n; i++) {  
    for (j = 0; j < n; j++)  
        a[ni + j] = b[j];  
    ni += n;  
}
```

Share Common Subexpressions

- Reuse portions of expressions
- GCC will do this with -O1

```
/* Sum neighbors of i,j */
up =    val[(i-1)*n + j  ];
down =  val[(i+1)*n + j  ];
left =  val[i*n          + j-1];
right = val[i*n          + j+1];
sum = up + down + left + right;
```

3 multiplications: $i*n$, $(i-1)*n$, $(i+1)*n$

```
leaq  1(%rsi), %rax  # i+1
leaq  -1(%rsi), %r8   # i-1
imulq %rcx, %rsi     # i*n
imulq %rcx, %rax     # (i+1)*n
imulq %rcx, %r8      # (i-1)*n
addq  %rdx, %rsi     # i*n+j
addq  %rdx, %rax     # (i+1)*n+j
addq  %rdx, %r8      # (i-1)*n+j
```

```
long inj = i*n + j;
up =    val[inj - n];
down =  val[inj + n];
left =  val[inj - 1];
right = val[inj + 1];
sum = up + down + left + right;
```

1 multiplication: $i*n$

```
imulq %rcx, %rsi  # i*n
addq  %rdx, %rsi  # i*n+j
movq  %rsi, %rax  # i*n+j
subq  %rcx, %rax  # i*n+j-n
leaq  (%rsi,%rcx), %rcx # i*n+j+n
```

Optimization Blocker #1: Procedure Calls

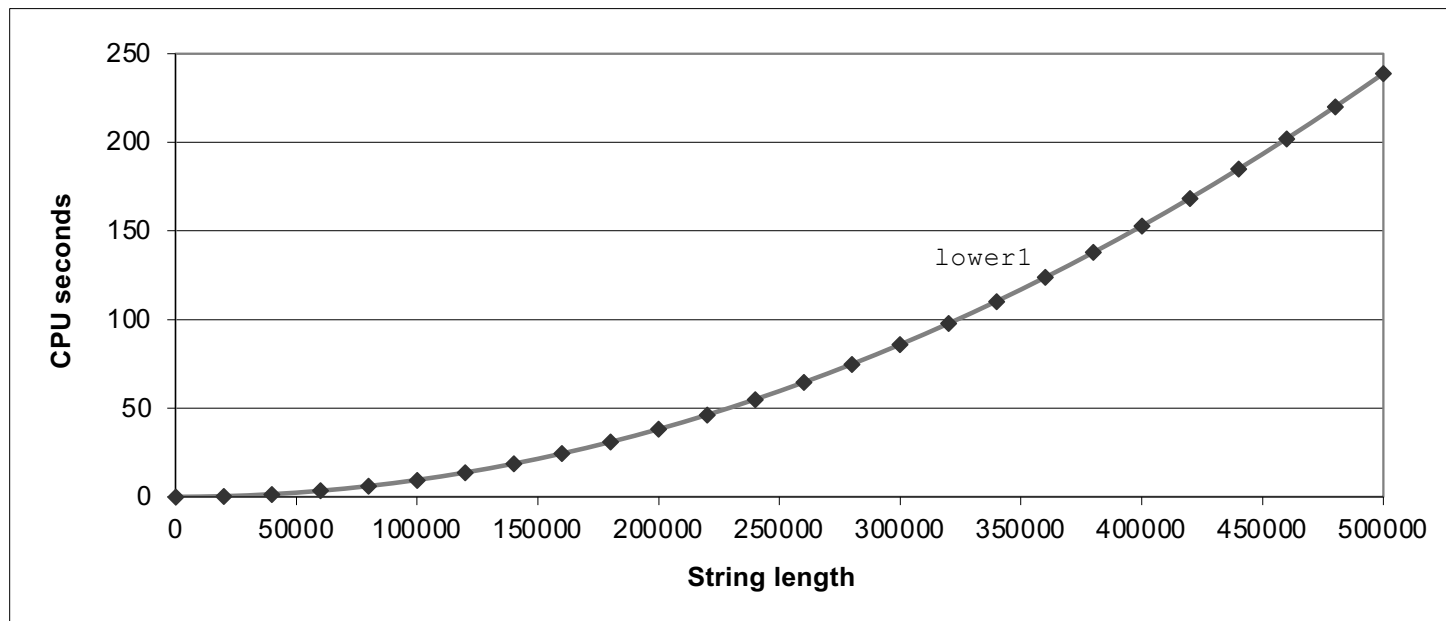
■ Procedure to Convert String to Lower Case

```
void lower(char *s)
{
    size_t i;
    for (i = 0; i < strlen(s); i++)
        if (s[i] >= 'A' && s[i] <= 'Z')
            s[i] -= ('A' - 'a');
}
```

0	00	NUL	25	19	EM	51	33	3	77	4D	M	103	67	g
1	01	SOH	26	1A	SUB	52	34	4	78	4E	N	104	68	h
2	02	STX	27	1B	ESC	53	35	5	79	4F	O	105	69	i
3	03	ETX	28	1C	FS	54	36	6	80	50	P	106	6A	j
4	04	EOT	29	1D	GS	55	37	7	81	51	Q	107	6B	k
5	05	ENQ	30	1E	RS	56	38	8	82	52	R	108	6C	l
6	06	ACK	31	1F	US	57	39	9	83	53	S	109	6D	m
7	07	BEL	32	20	space	58	3A	:	84	54	T	110	6E	n
8	08	BS	33	21	!	59	3B	;	85	55	U	111	6F	o
9	09	HT	34	22	"	60	3C	<	86	56	V	112	70	p
10	0A	LF	35	23	#	61	3D	=	87	57	W	113	71	q
11	0B	VT	36	24	\$	62	3E	>	88	58	X	114	72	r
12	0C	FF	37	25	%	63	3F	?	89	59	Y	115	73	s
13	0D	CR	38	26	&	64	40	@	90	5A	Z	116	74	t
14	0E	SO	39	27	'	65	41	A	91	5B	[	117	75	u
15	0F	SI	40	28	(	66	42	B	92	5C	\	118	76	v
16	10	DLE	41	29	)	67	43	C	93	5D	]	119	77	w
17	11	DC1	42	2A	*	68	44	D	94	5E	^	120	78	x
18	12	DC2	43	2B	+	69	45	E	95	5F	_	121	79	y
19	13	DC3	44	2C	,	70	46	F	96	60	`	122	7A	z
20	14	DC4	45	2D	-	71	47	G	97	61	a	123	7B	{
21	15	NAK	46	2E	.	72	48	H	98	62	b	124	7C	
22	16	SYN	47	2F	/	73	49	I	99	63	c	125	7D	}
23	17	ETB	48	30	0	74	4A	J	100	64	d	126	7E	~
24	18	CAN	49	31	1	75	4B	K	101	65	e	127	7F	DEL
			50	32	2	76	4C	L	102	66	f			

Lower Case Conversion Performance

- Time quadruples when double string length
- Quadratic performance



Convert Loop To Goto Form

```
void lower(char *s)
{
    size_t i = 0;
    if (i >= strlen(s))
        goto done;
loop:
    if (s[i] >= 'A' && s[i] <= 'Z')
        s[i] -= ('A' - 'a');
    i++;
    if (i < strlen(s))
        goto loop;
done:
}
```

- `strlen` executed every iteration

Calling Strlen

```
/* My version of strlen */
size_t strlen(const char *s)
{
    size_t length = 0;
    while (*s != '\0') {
        s++;
        length++;
    }
    return length;
}
```

■ Strlen performance

- Only way to determine length of string is to scan its entire length, looking for null character.

■ Overall performance, string of length N

- N calls to strlen
- Require times N, N-1, N-2, ..., 1
- Overall $O(N^2)$ performance

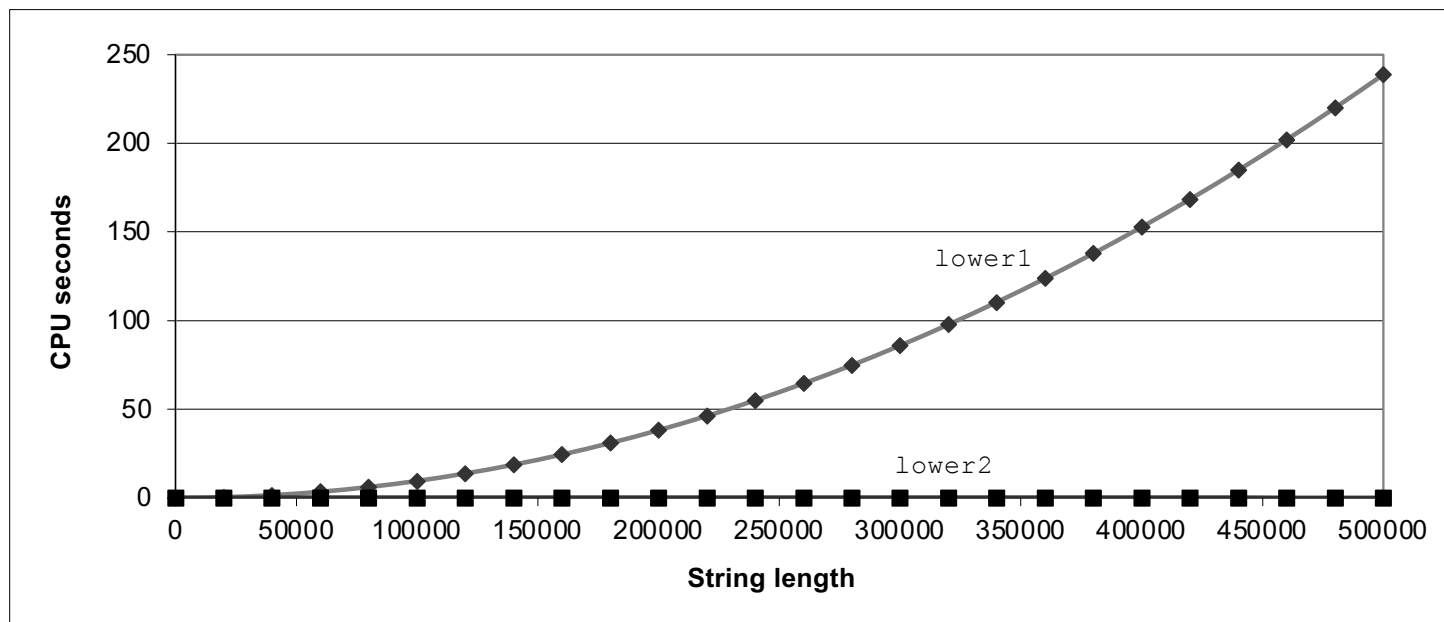
Improving Performance

```
void lower(char *s)
{
    size_t i;
    size_t len = strlen(s);
    for (i = 0; i < len; i++)
        if (s[i] >= 'A' && s[i] <= 'Z')
            s[i] -= ('A' - 'a');
}
```

- Move call to `strlen` outside of loop
- Since result does not change from one iteration to another
- Form of code motion

Lower Case Conversion Performance

- Time doubles when double string length
- Linear performance of lower2



Optimization Blocker: Procedure Calls

■ *Why couldn't compiler move `strlen` out of inner loop?*

- Procedure may have side effects
 - Alters global state each time called
- Function may not return same value for given arguments
 - Depends on other parts of global state
 - Procedure `lower` could interact with `strlen`

■ **Warning:**

- Compiler treats procedure call as a black box
- Weak optimizations near them

■ **Remedies:**

- Use of inline functions
 - GCC does this with `-O1`
 - Within single file
- Do your own code motion

```
size_t lencnt = 0;
size_t strlen(const char *s)
{
    size_t length = 0;
    while (*s != '\0') {
        s++; length++;
    }
    lencnt += length;
    return length;
}
```

Memory Matters

```
/* Sum rows is of n X n matrix a
   and store in vector b */
void sum_rows1(double *a, double *b, long n) {
    long i, j;
    for (i = 0; i < n; i++) {
        b[i] = 0;
        for (j = 0; j < n; j++)
            b[i] += a[i*n + j];
    }
}
```

```
# sum_rows1 inner loop
.L4:
    movsd    (%rsi,%rax,8), %xmm0    # FP load
    addsd    (%rdi), %xmm0           # FP add
    movsd    %xmm0, (%rsi,%rax,8)    # FP store
    addq     $8, %rdi
    cmpq     %rcx, %rdi
    jne      .L4
```

- Code updates `b[i]` on every iteration
- Why couldn't compiler optimize this away?

Memory Aliasing

```
/* Sum rows is of n X n matrix a
   and store in vector b */
void sum_rows1(double *a, double *b, long n) {
    long i, j;
    for (i = 0; i < n; i++) {
        b[i] = 0;
        for (j = 0; j < n; j++)
            b[i] += a[i*n + j];
    }
}
```

```
double A[9] =
{ 0, 1, 2,
  4, 8, 16,
  32, 64, 128};

double *B = A+3;

sum_rows1(A, B, 3);
```

Value of B:

init: [4, 8, 16]

i = 0: [3, 8, 16]

i = 1: [3, 22, 16]

i = 2: [3, 22, 224]

- Code updates `b[i]` on every iteration
- Must consider possibility that these updates will affect program behavior

Removing Aliasing

```
/* Sum rows is of n X n matrix a
   and store in vector b */
void sum_rows2(double *a, double *b, long n) {
    long i, j;
    for (i = 0; i < n; i++) {
        double val = 0;
        for (j = 0; j < n; j++)
            val += a[i*n + j];
        b[i] = val;
    }
}
```

```
# sum_rows2 inner loop
.L10:
    addsd    (%rdi), %xmm0    # FP load + add
    addq     $8, %rdi
    cmpq     %rax, %rdi
    jne      .L10
```

- No need to store intermediate results

Optimization Blocker: Memory Aliasing

■ Aliasing

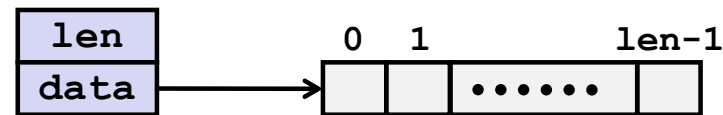
- Two different memory references specify single location
- Easy to have happen in C
 - Since allowed to do address arithmetic
 - Direct access to storage structures
- Get in habit of introducing local variables
 - Accumulating within loops
 - **Your way of telling compiler not to check for aliasing**

Exploiting Instruction-Level Parallelism

- **Need general understanding of modern processor design**
 - Hardware can execute multiple instructions in parallel
- **Performance limited by data dependencies**
- **Simple transformations can yield dramatic performance improvement**
 - Compilers often cannot make these transformations
 - Lack of associativity and distributivity in floating-point arithmetic

Benchmark Example: Data Type for Vectors

```
/* data structure for vectors */
typedef struct{
    size_t len;
    data_t *data;
} vec;
```



■ Data Types

- Use different declarations for data_t
- int
- long
- float
- double

```
/* retrieve vector element
and store at val */
int get_vec_element
(*vec v, size_t idx, data_t *val)
{
    if (idx >= v->len)
        return 0;
    *val = v->data[idx];
    return 1;
}
```

Benchmark Computation

```
void combin1(vec_ptr v, data_t *dest)
{
    long int i;
    *dest = IDENT;
    for (i = 0; i < vec_length(v); i++) {
        data_t val;
        get_vec_element(v, i, &val);
        *dest = *dest OP val;
    }
}
```

**Compute sum or
product of vector
elements**

■ Data Types

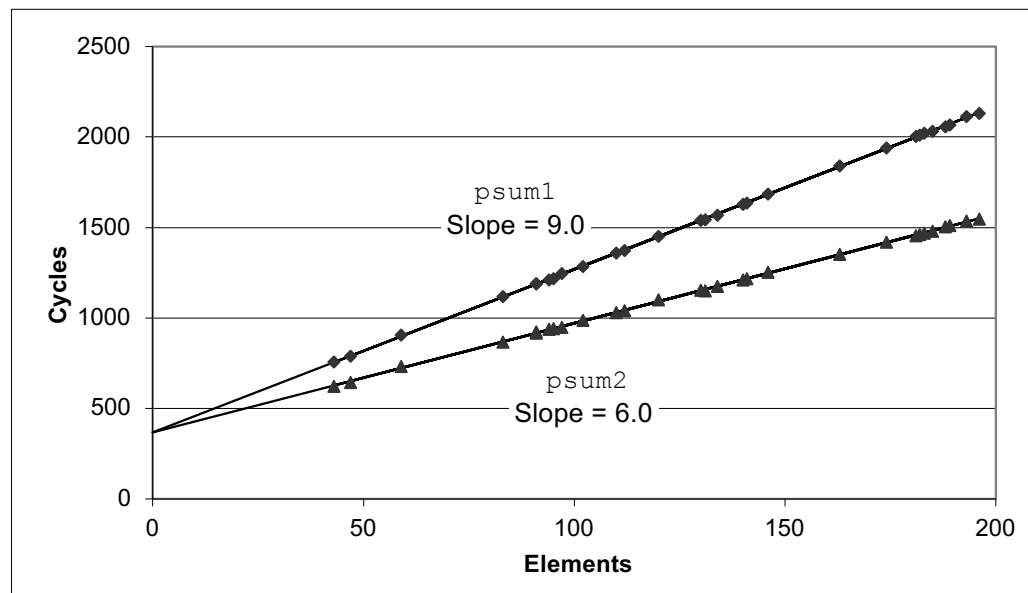
- Use different declarations for data_t
- int
- long
- float
- double

■ Operations

- Use different definitions of OP and IDENT
- + / 0
- * / 1

Cycles Per Element (CPE)

- Convenient way to express performance of program that operates on vectors or lists
- Length = n
- In our case: **CPE = cycles per OP**
- $T = CPE * n + \text{Overhead}$
 - CPE is slope of line



Benchmark Performance

```
void combine1(vec_ptr v, data_t *dest)
{
    long int i;
    *dest = IDENT;
    for (i = 0; i < vec_length(v); i++) {
        data_t val;
        get_vec_element(v, i, &val);
        *dest = *dest OP val;
    }
}
```

Compute sum or
product of vector
elements

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine1 unoptimized	22.68	20.02	19.98	20.18
Combine1 -O1	10.12	10.12	10.17	11.14

Basic Optimizations

```
void combine4(vec_ptr v, data_t *dest)
{
    long i;
    long length = vec_length(v);
    data_t *d = get_vec_start(v);
    data_t t = IDENT;
    for (i = 0; i < length; i++)
        t = t OP d[i];
    *dest = t;
}
```

- Move `vec_length` out of loop
- Avoid bounds check on each cycle
- Accumulate in temporary

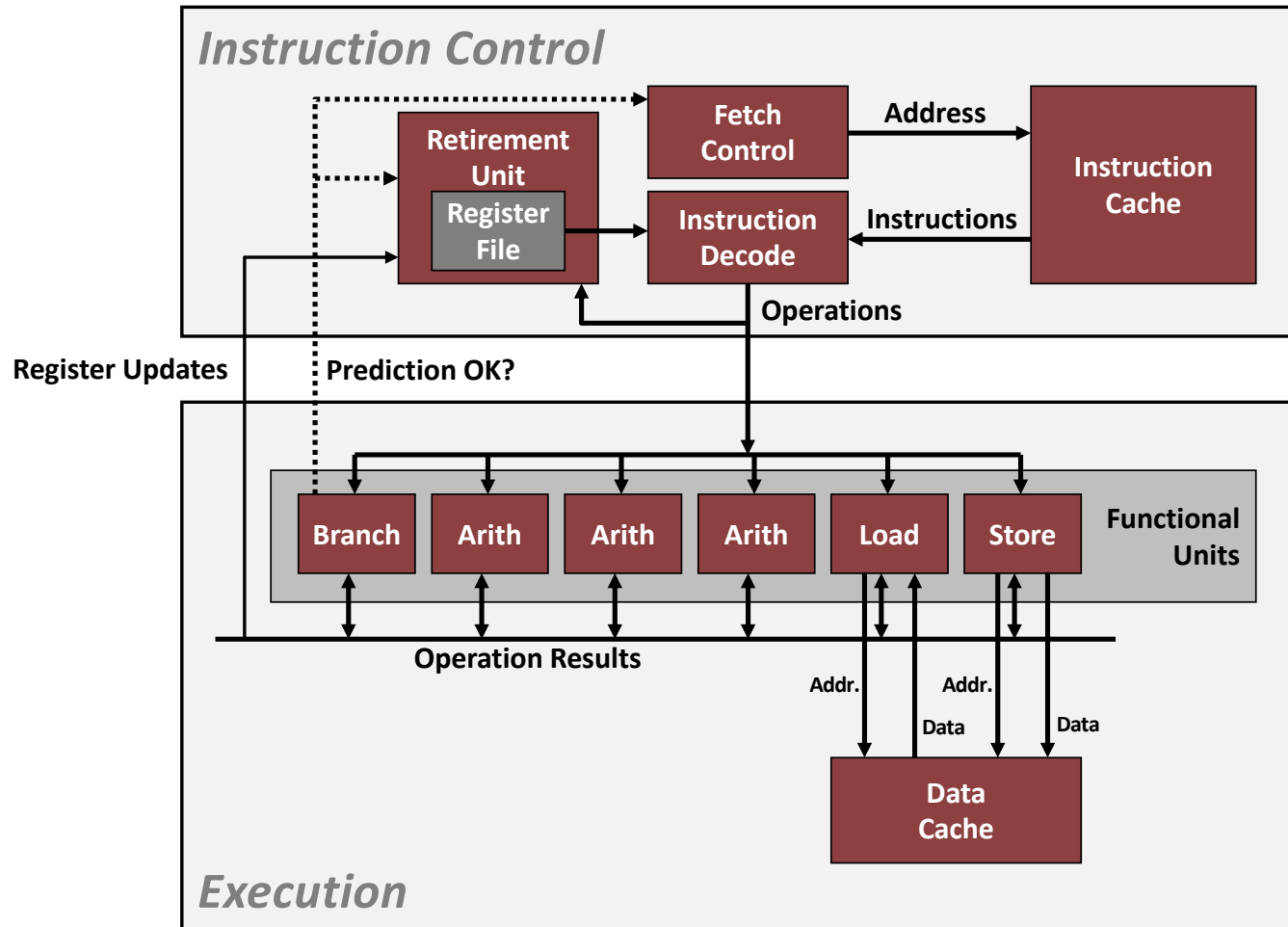
Effect of Basic Optimizations

```
void combine4(vec_ptr v, data_t *dest)
{
    long i;
    long length = vec_length(v);
    data_t *d = get_vec_start(v);
    data_t t = IDENT;
    for (i = 0; i < length; i++)
        t = t OP d[i];
    *dest = t;
}
```

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine1 -O1	10.12	10.12	10.17	11.14
Combine4	1.27	3.01	3.01	5.01

- Eliminates sources of overhead in loop

Modern CPU Design

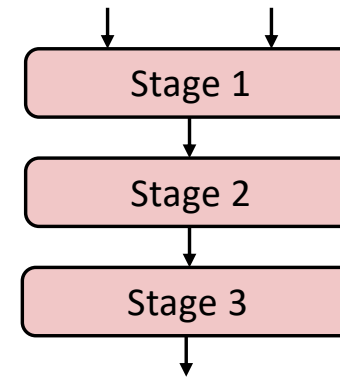


Superscalar Processor

- **Definition:** A superscalar processor can issue and execute *multiple instructions in one cycle*. The instructions are retrieved from a sequential instruction stream and are usually scheduled dynamically.
- **Benefit:** without programming effort, superscalar processor can take advantage of the *instruction level parallelism* that most programs have
- Most modern CPUs are superscalar.
- Intel: since Pentium (1993)

Pipelined Functional Units

```
long mult_eg(long a, long b, long c) {  
    long p1 = a * b;  
    long p2 = a * c;  
    long p3 = p1 * p2;  
    return p3;  
}
```



Time							
	1	2	3	4	5	6	7
Stage 1	a*b	a*c			p1*p2		
Stage 2		a*b	a*c			p1*p2	
Stage 3			a*b	a*c			p1*p2

- Divide computation into stages
- Pass partial computations from stage to stage
- Stage i can start on new computation once values passed to i+1
- E.g., complete 3 multiplications in 7 cycles, even though each requires 3 cycles

Haswell CPU

■ 8 Total Functional Units

■ Multiple instructions can execute in parallel

1. Integer arithmetic, floating-point multiplication, integer and floating-point division, branches
2. Integer arithmetic, floating-point addition, integer multiplication, floating-point multiplication
3. Load, address computation
4. Load, address computation
5. Store
6. Integer arithmetic
7. Integer arithmetic, branches
8. Store address computation

4个功能单元能执行整数操作

2个功能单元能执行加载操作

2个功能单元能执行浮点乘法

1个功能单元能执行浮点加法操作

Haswell CPU

■ Some instructions take > 1 cycle, but can be pipelined

<i>Instruction</i>	<i>Latency</i>	<i>Cycles/Issue</i>	<i>Capacity</i>
Integer Add	1	1	4
Integer Multiply	3	1	1
Integer/Long Divide	3-30	3-30	1
Single/Double FP Add	3	1	1
Single/Double FP Multiply	5	1	2
Single/Double FP Divide	3-15	3-15	1

latency 延迟：完成运算所需要的总时间

issue time 发射时间**I**：两个连续的同类型运算之间需要的最小时钟周期数。

发射时间为**1**的功能单元被称为完全流水线化的（**fully pipelined**）。

capacity 容量**C**：能够执行该运算的功能单元数量

吞吐量： C/I

x86-64 Compilation of Combine4

■ Inner Loop (Case: Integer Multiply)

```
.L519:                # Loop:
    imull  (%rax,%rdx,4), %ecx  # t = t * d[i]
    addq   $1, %rdx            # i++
    cmpq   %rdx, %rbp          # Compare length:i
    jg     .L519               # If >, goto Loop
```

■ Inner Loop (Case: Double Multiply)

```
.L25:
    vmulsd (%rdx), %xmm0, %xmm0 # Multiply acc by data[i]
    addq   $8, %rdx             # Increment data+i
    cmpq   %rax, %rdx           # Compare to data+length
    jne    .L25                 # If !=, goto loop
```

```
void combine4(vec_ptr v, data_t *dest)
{
    long i;
    long length = vec_length(v);
    data_t *d = get_vec_start(v);
    data_t t = IDENT;
    for (i = 0; i < length; i++)
        t = t OP d[i];
    *dest = t;
}
```

Graphical Representation of Combine4

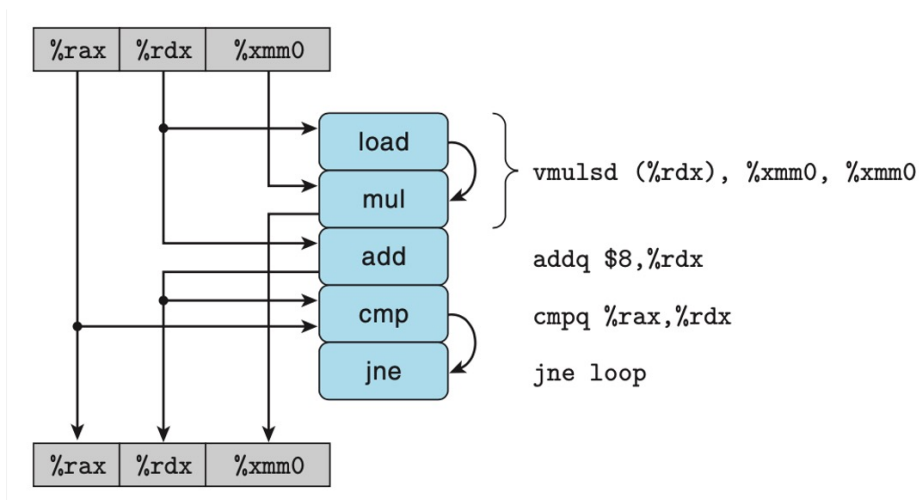


Fig. 5.13 Graphical representation of inner-loop code for combine4.

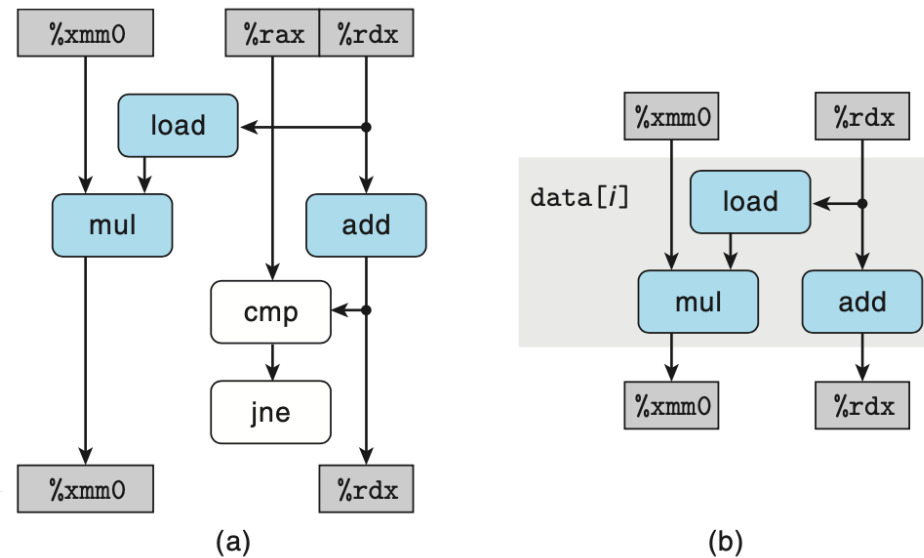


Fig. 5.14 Abstracting combine4 operations as a data-flow graph.

Graphical Representation of Combine4

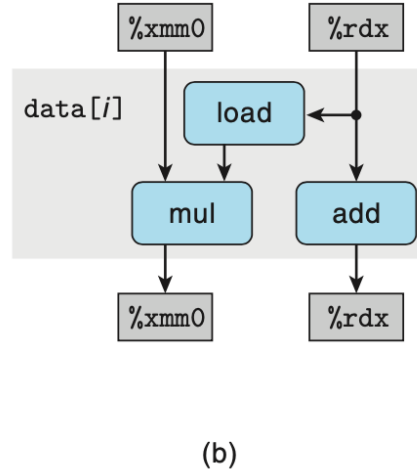
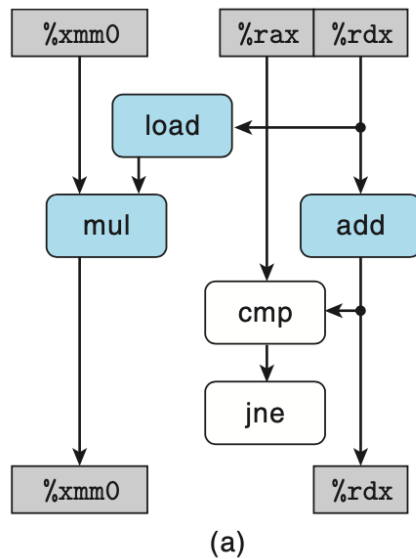
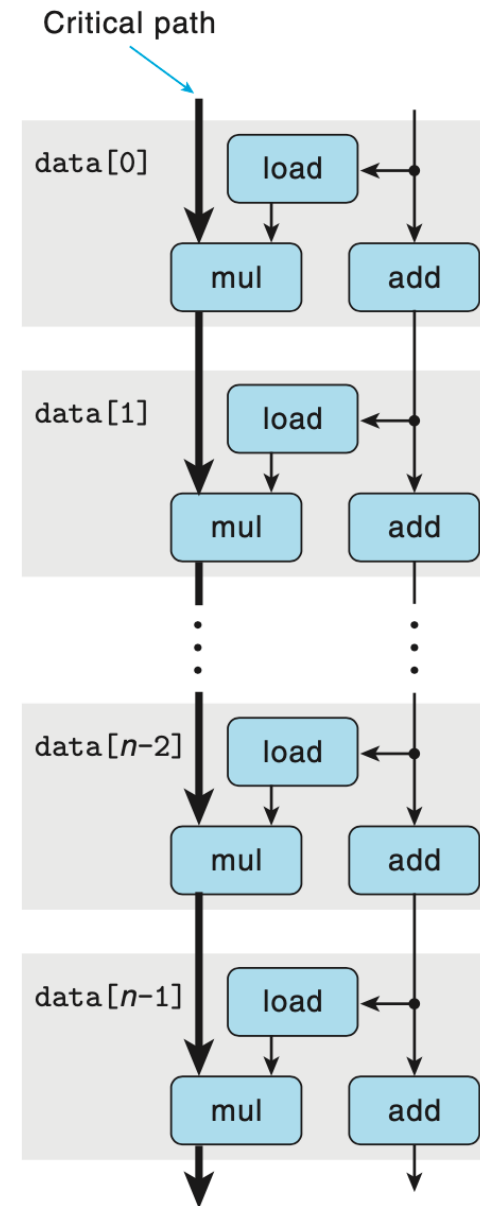
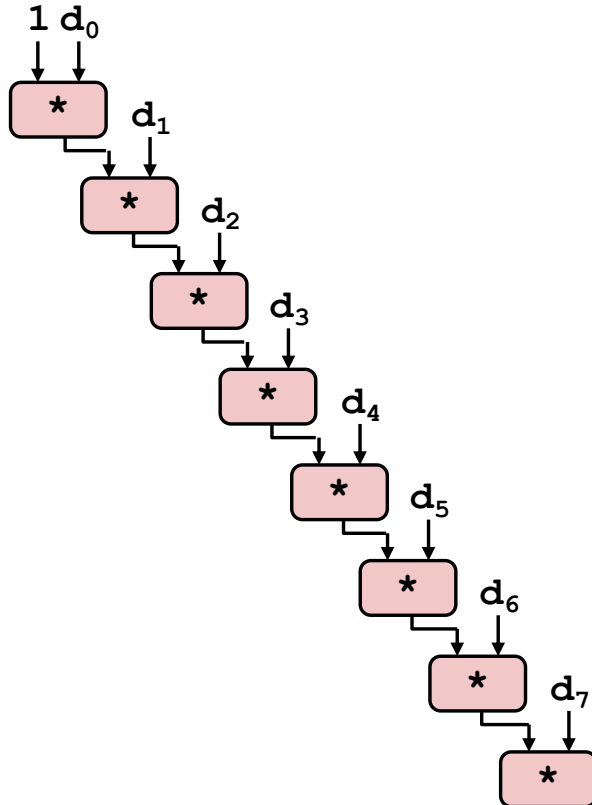


Fig. 5.14 Abstracting combine4 operations as a data-flow graph.

Fig. 5.15 Data-flow representation of computation by n iterations of the inner loop of combine4.



Combine4 = Serial Computation (OP = *)



■ Computation (length=8)

$(((((1 * d[0]) * d[1]) * d[2]) * d[3]) * d[4]) * d[5]) * d[6]) * d[7])$

■ Sequential dependence

- Performance: determined by latency of OP

Method	Integer		Double FP	
	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
Latency Bound	1.00	3.00	3.00	5.00

Loop Unrolling (2x1)

```
void unroll2a_combine(vec_ptr v, data_t *dest)
{
    long length = vec_length(v);
    long limit = length-1;
    data_t *d = get_vec_start(v);
    data_t x = IDENT;
    long i;
    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        x = (x OP d[i]) OP d[i+1];
    }
    /* Finish any remaining elements */
    for (; i < length; i++) {
        x = x OP d[i];
    }
    *dest = x;
}
```

- Perform 2x more useful work per iteration

Effect of Loop Unrolling

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
Unroll 2x1	1.01	3.01	3.01	5.01
Latency Bound	1.00	3.00	3.00	5.00

- **Helps integer add**
 - Achieves latency bound
- **Others don't improve. *Why?***
 - Still sequential dependency

```
x = (x OP d[i]) OP d[i+1];
```

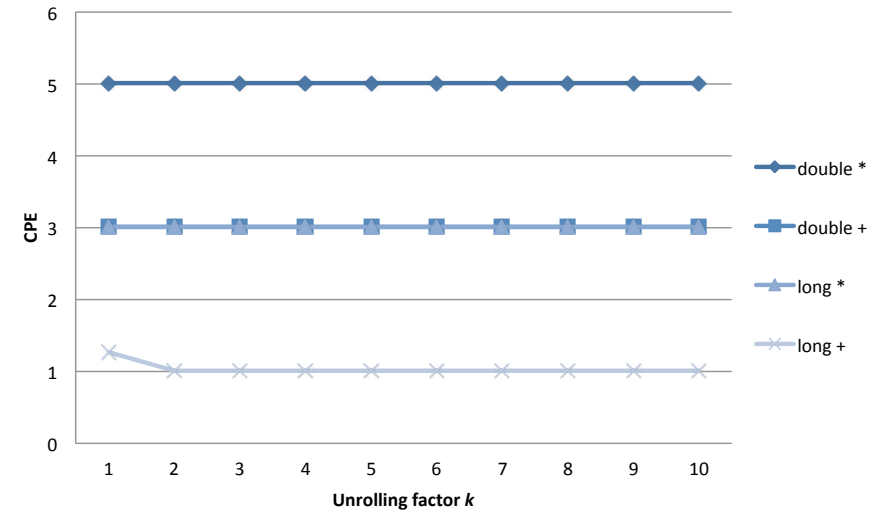


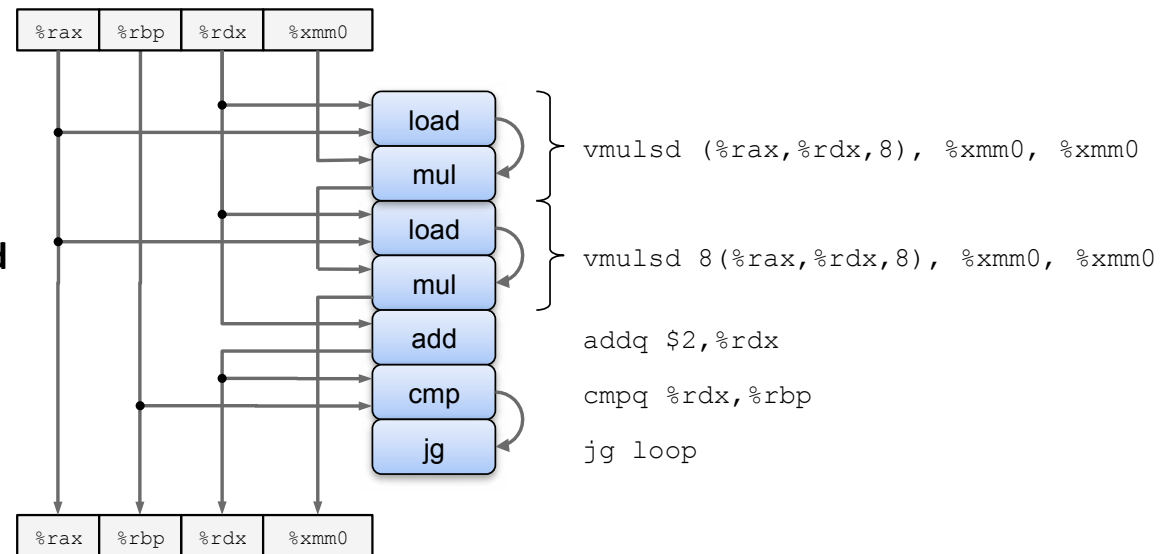
Fig. 5.17 CPE performance for difference degrees of k x 1 loop unrolling. Only integer addition improves CPE with this transformation.

Loop Unrolling: Combine5

Inner loop of combine5. data_t = double, OP = * i in %rdx, data %rax, limit in %rbp, acc in %xmm0

```
.L35:                                #loop:
    vmulsd (%rax,%rdx,8), %xmm0, %xmm0  #Multiply acc by data[i]
    vmulsd 8(%rax,%rdx,8), %xmm0, %xmm0 #Multiply acc by data[i+1]
    addq    $2, %rdx                    #Increment i by 2
    cmpq    %rdx, %rbp                  #Compare to limit:i
    jg      .L35                        #If >, goto loop
```

Fig. 5.18 Graphical representation of inner-loop code for combine5.



Graphical Representation of Combine5

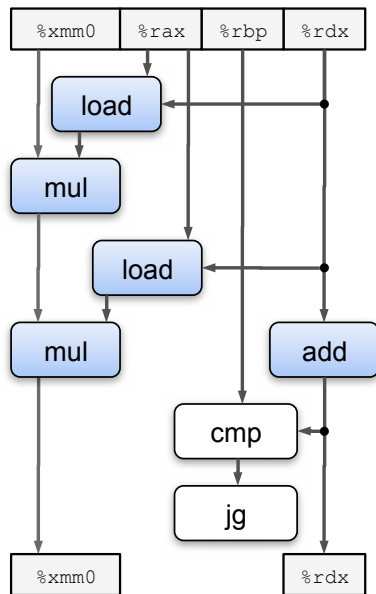


Fig. 5.19 Abstracting combine5 operations as a data-flow graph.

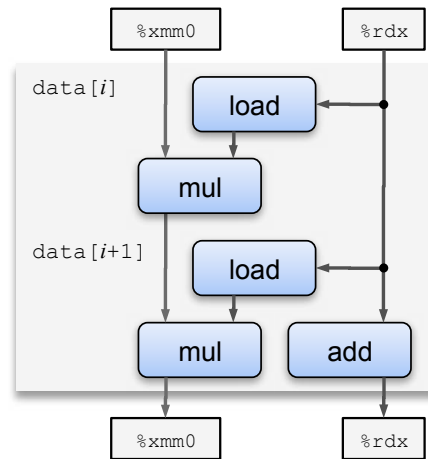
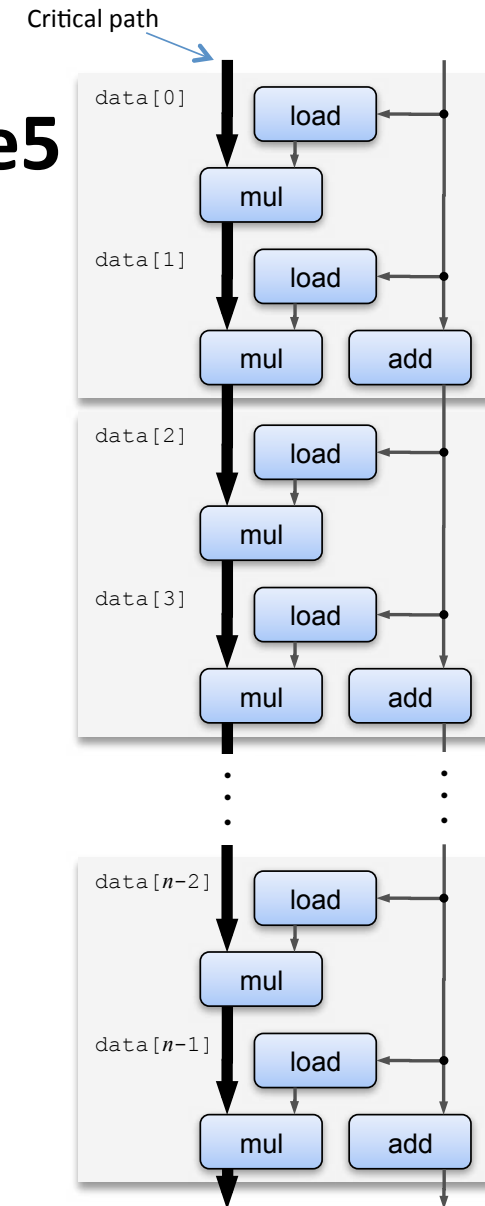


Fig. 5.20 Data-flow representation of combine5 operating on a vector of length n .



Loop Unrolling with Reassociation (2x1a)

```
void unroll2aa_combine(vec_ptr v, data_t *dest)
{
    long length = vec_length(v);
    long limit = length-1;
    data_t *d = get_vec_start(v);
    data_t x = IDENT;
    long i;
    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        x = x OP (d[i] OP d[i+1]);
    }
    /* Finish any remaining elements */
    for (; i < length; i++) {
        x = x OP d[i];
    }
    *dest = x;
}
```

Compare to before

```
x = (x OP d[i]) OP d[i+1];
```

- Can this change the result of the computation?
- Yes, for FP. *Why?*

Effect of Reassociation

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
Unroll 2x1	1.01	3.01	3.01	5.01
Unroll 2x1a	1.01	1.51	1.51	2.51
Latency Bound	1.00	3.00	3.00	5.00
Throughput Bound	0.50	1.00	1.00	0.50

■ Nearly 2x speedup for Int *, FP +, FP *

- Reason: Breaks sequential dependency

```
x = x OP (d[i] OP d[i+1]);
```

- Why is that?

1 func. units for int *
2 func. units for load

2 func. units for FP *
2 func. units for load

1 func. units for FP +
2 func. units for load

4 func. units for int +
2 func. units for load

Graphical Representation of Combine7

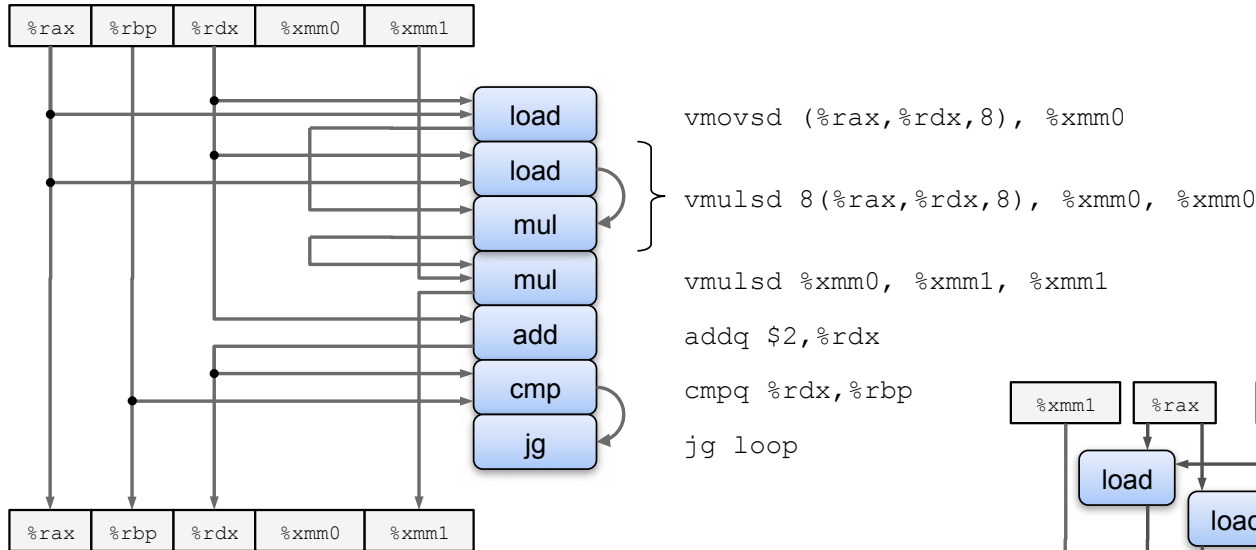


Fig. 5.27 Graphical representation of inner-loop code for combine7.

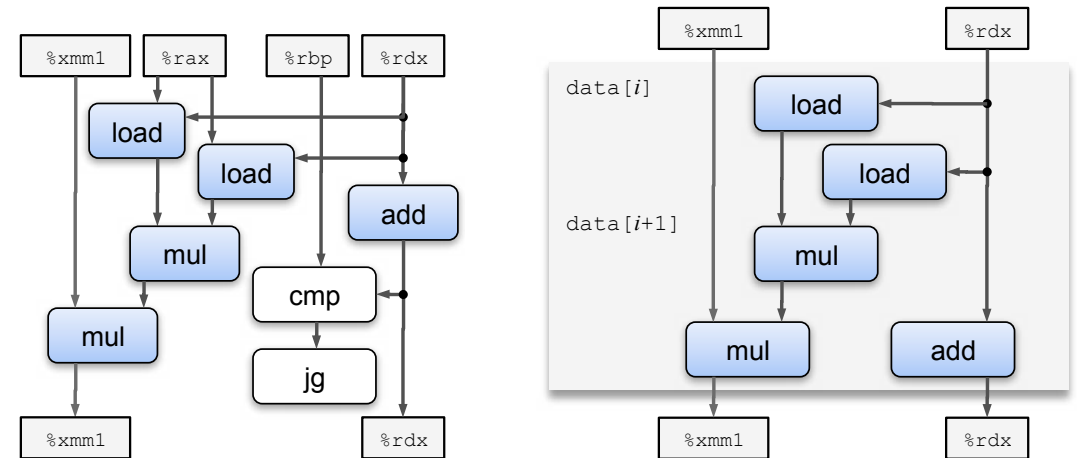
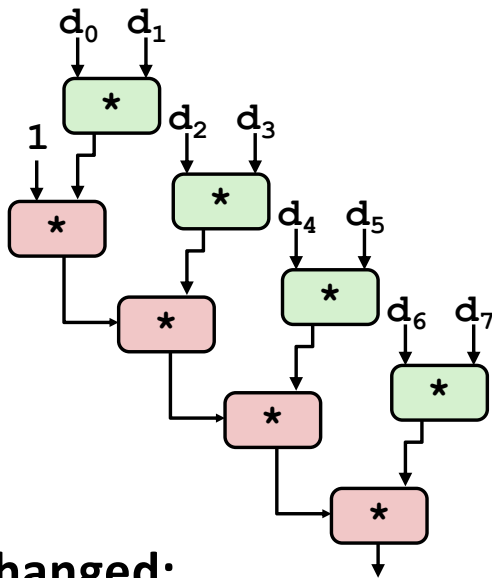


Fig. 5.28 Abstracting combine7 operations as a data-flow graph.

Reassociated Computation

```
x = x OP (d[i] OP d[i+1]);
```



■ What changed:

- Ops in the next iteration can be started early (no dependency)

■ Overall Performance

- N elements, D cycles latency/op
- $(N/2+1)*D$ cycles:
CPE = D/2

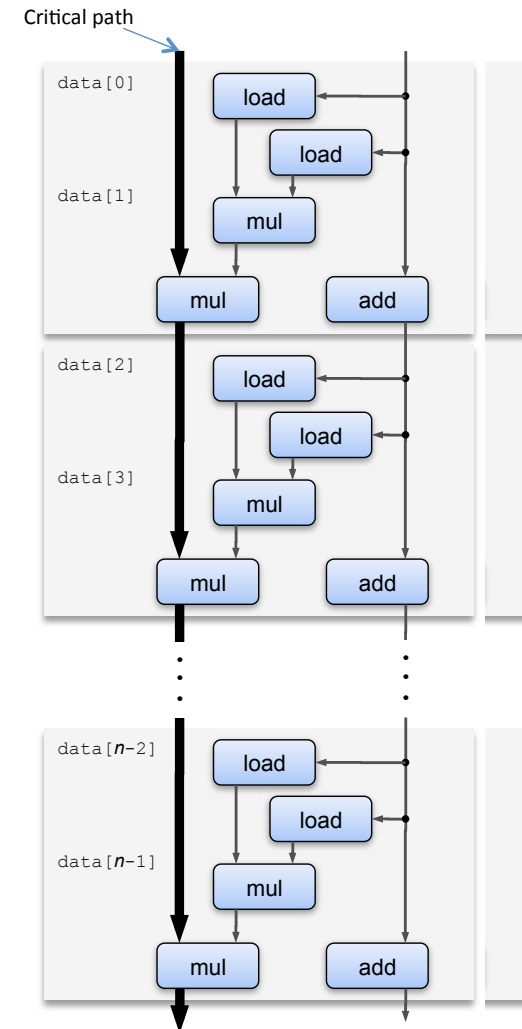


Fig. 5.29 Data-flow representation of combine7 operating on a vector of length n. We have a single critical path, but it contains only $n/2$ operations.

Effect of Reassociation

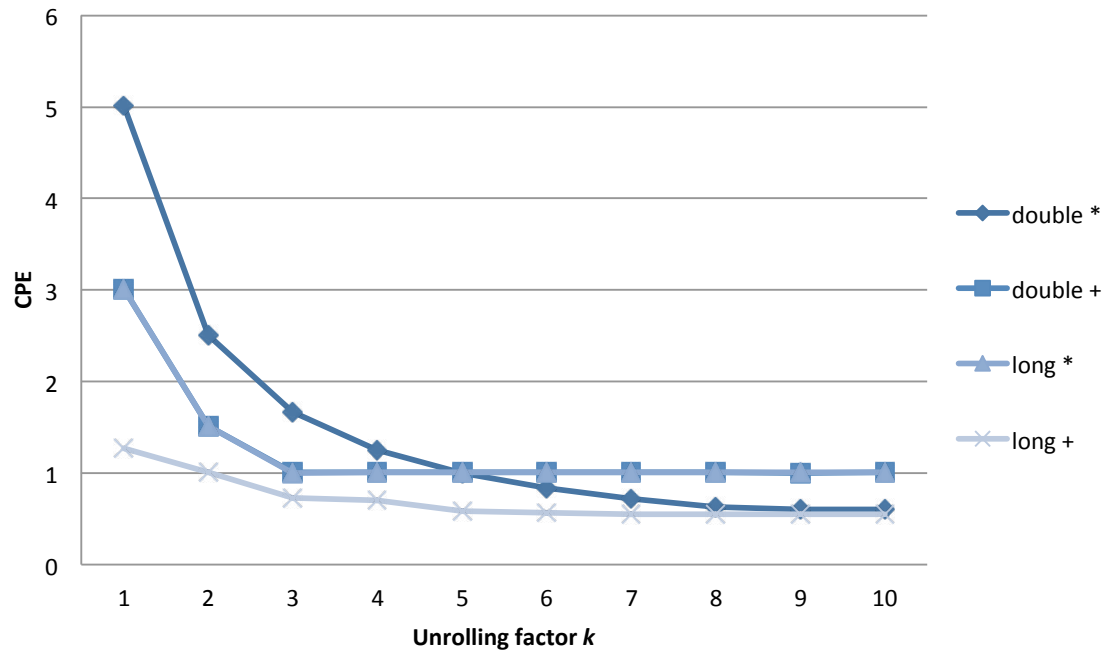


Fig. 5.30 CPE performance for $k \times 1a$ loop unrolling. All of the CPEs improve with this transformation, nearly approaching their throughput bounds.

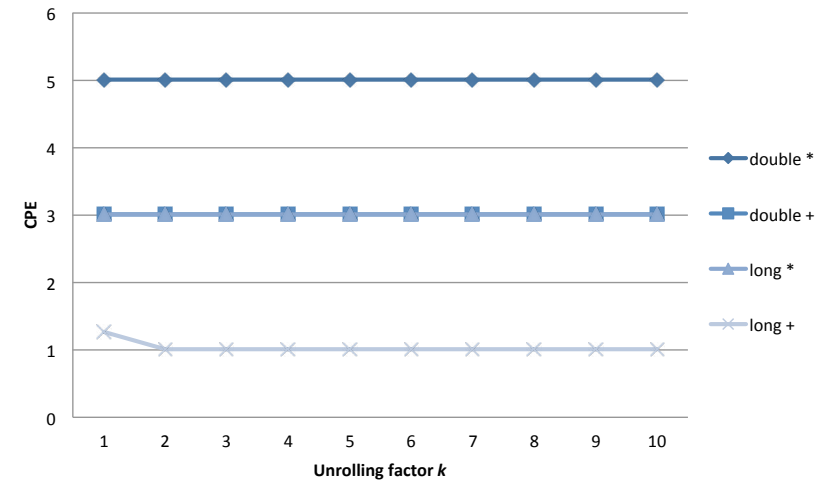


Fig. 5.17 CPE performance for difference degrees of $k \times 1$ loop unrolling. Only integer addition improves CPE with this transformation.

Loop Unrolling with Separate Accumulators (2x2)

```
void unroll2a_combine(vec_ptr v, data_t *dest)
{
    long length = vec_length(v);
    long limit = length-1;
    data_t *d = get_vec_start(v);
    data_t x0 = IDENT;
    data_t x1 = IDENT;
    long i;
    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        x0 = x0 OP d[i];
        x1 = x1 OP d[i+1];
    }
    /* Finish any remaining elements */
    for (; i < length; i++) {
        x0 = x0 OP d[i];
    }
    *dest = x0 OP x1;
}
```

■ Different form of reassociation

Effect of Separate Accumulators

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine4	1.27	3.01	3.01	5.01
Unroll 2x1	1.01	3.01	3.01	5.01
Unroll 2x1a	1.01	1.51	1.51	2.51
Unroll 2x2	0.81	1.51	1.51	2.51
Latency Bound	1.00	3.00	3.00	5.00
Throughput Bound	0.50	1.00	1.00	0.50

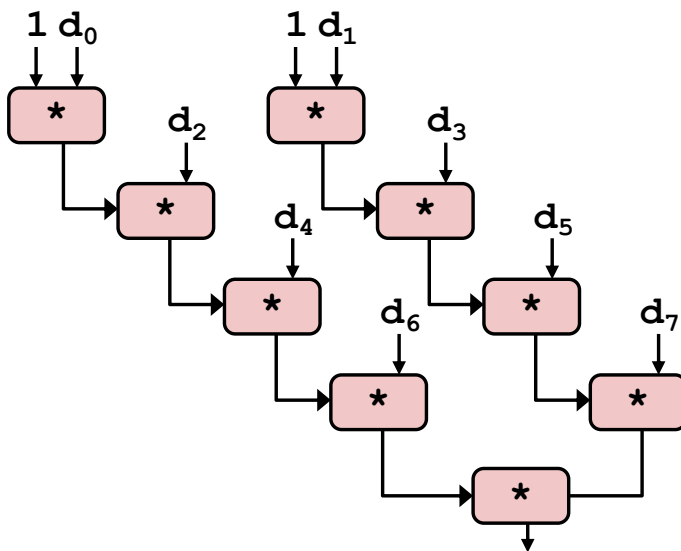
- Int + makes use of two load units

```
x0 = x0 OP d[i];
x1 = x1 OP d[i+1];
```

- 2x speedup (over unroll 2x1) for Int *, FP +, FP *

Separate Accumulators

```
x0 = x0 OP d[i];  
x1 = x1 OP d[i+1];
```



■ What changed:

- Two independent “streams” of operations

■ Overall Performance

- N elements, D cycles latency/op
- Should be $(N/2+1)*D$ cycles:
 $CPE = D/2$
- CPE matches prediction!

What Now?

Unrolling & Accumulating

■ Idea

- Can unroll to any degree L
- Can accumulate K results in parallel
- L must be multiple of K

■ Limitations

- Diminishing returns
 - Cannot go beyond throughput limitations of execution units
- Large overhead for short lengths
 - Finish off iterations sequentially

Unrolling & Accumulating: Double *

■ Case

- Intel Haswell
- Double FP Multiplication
- Latency bound: 5.00. Throughput bound: 0.50

Accumulators	FP *	Unrolling Factor L							
	K	1	2	3	4	6	8	10	12
	1	5.01	5.01	5.01	5.01	5.01	5.01	5.01	
	2		2.51		2.51		2.51		
	3			1.67					
	4				1.25		1.26		
	6					0.84			0.88
	8						0.63		
	10							0.51	
	12								0.52

Unrolling & Accumulating: Int +

■ Case

- Intel Haswell
- Integer addition
- Latency bound: 1.00. Throughput bound: 0.50

Accumulators	FP *	Unrolling Factor L							
	K	1	2	3	4	6	8	10	12
	1	1.27	1.01	1.01	1.01	1.01	1.01	1.01	
	2		0.81		0.69		0.54		
	3			0.74					
	4				0.69		1.24		
	6					0.56			0.56
	8						0.54		
	10							0.54	
	12								0.56

Achievable Performance

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Best	0.54	1.01	1.01	0.52
Latency Bound	1.00	3.00	3.00	5.00
Throughput Bound	0.50	1.00	1.00	0.50

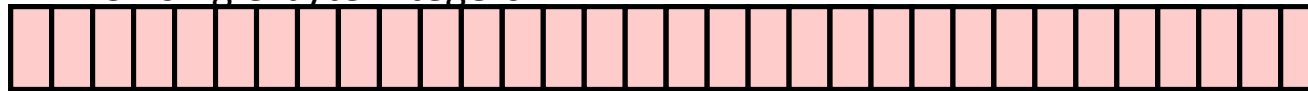
- Limited only by throughput of functional units
- Up to 42X improvement over original, unoptimized code

Programming with AVX2

YMM Registers

■ 16 total, each 32 bytes

■ 32 single-byte integers



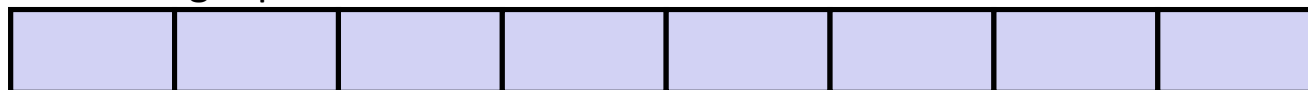
■ 16 16-bit integers



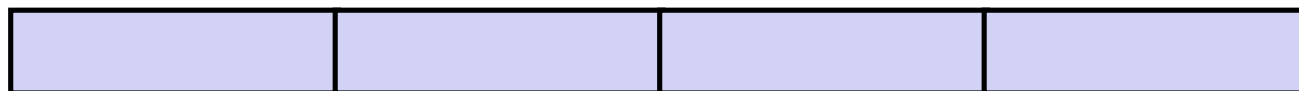
■ 8 32-bit integers



■ 8 single-precision floats



■ 4 double-precision floats



■ 1 single-precision float

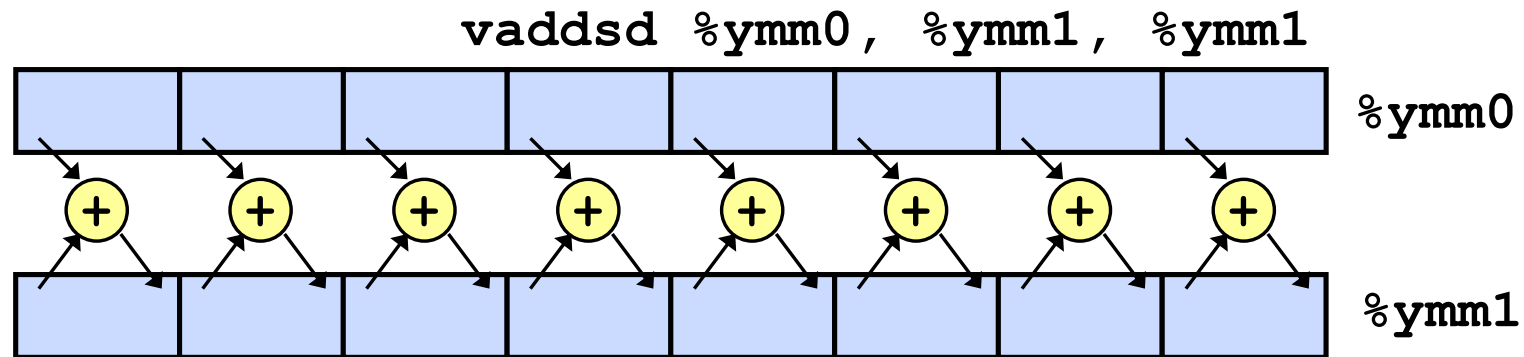


■ 1 double-precision float

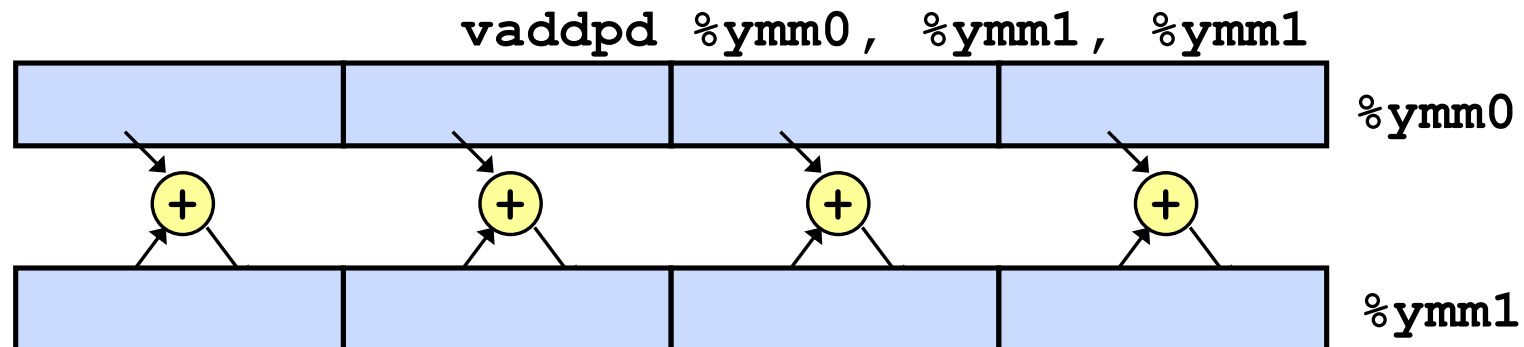


SIMD Operations

■ SIMD Operations: Single Precision



■ SIMD Operations: Double Precision



Using Vector Instructions

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Scalar Best	0.54	1.01	1.01	0.52
Vector Best	0.06	0.24	0.25	0.16
Latency Bound	0.50	3.00	3.00	5.00
Throughput Bound	0.50	1.00	1.00	0.50
Vec Throughput Bound	0.06	0.12	0.25	0.12

■ Make use of AVX Instructions

- Parallel operations on multiple data elements
- See Web Aside OPT:SIMD on CS:APP web page

There's plenty of room at the Top: What will drive computer performance after Moore's law?

Leiserson et al., Science 368, 1079 (2020), 5 June 2020

ming language. Here is the four-line kernel of
the Python 2 code for matrix-multiplication:

```
for i in xrange(4096):
    for j in xrange(4096):
        for k in xrange(4096):
            C[i][j] += A[i][k] * B[k][j]
```

Table 1. Speedups from performance engineering a program that multiplies two 4096-by-4096 matrices. Each version represents a successive refinement of the original Python code. "Running time" is the running time of the version. "GFLOPS" is the billions of 64-bit floating-point operations per second that the version executes. "Absolute speedup" is time relative to Python, and "relative speedup," which we show with an additional digit of precision, is time relative to the preceding line. "Fraction of peak" is GFLOPS relative to the computer's peak 835 GFLOPS. See Methods for more details.

Version	Implementation	Running time (s)	GFLOPS	Absolute speedup	Relative speedup	Fraction of peak (%)
1	Python	25,552.48	0.005	1	—	0.00
2	Java	2,372.68	0.058	11	10.8	0.01
3	C	542.67	0.253	47	4.4	0.03
4	Parallel loops	69.80	1.969	366	7.8	0.24
5	Parallel divide and conquer	3.80	36.180	6,727	18.4	4.33
6	plus vectorization	1.10	124.914	23,224	3.5	14.96
7	plus AVX intrinsics	0.41	337.812	62,806	2.7	40.45

Some Limiting Factors

■ Register Spilling

Function	Page	Method	Integer		Floating point	
			+	*	+	*
combine6	537					
		10 × 10 unrolling	0.55	1.00	1.01	0.52
		20 × 20 unrolling	0.83	1.03	1.02	0.68
Throughput bound			0.50	1.00	1.00	0.50

Register Spilling

- Inner loop of the code with 10×10 unrolling

```
Updating of accumulator acc0 in 10 x 10 unrolling acc0 *= data[i]  
vmulsd (%rdx), %xmm0, %xmm0
```

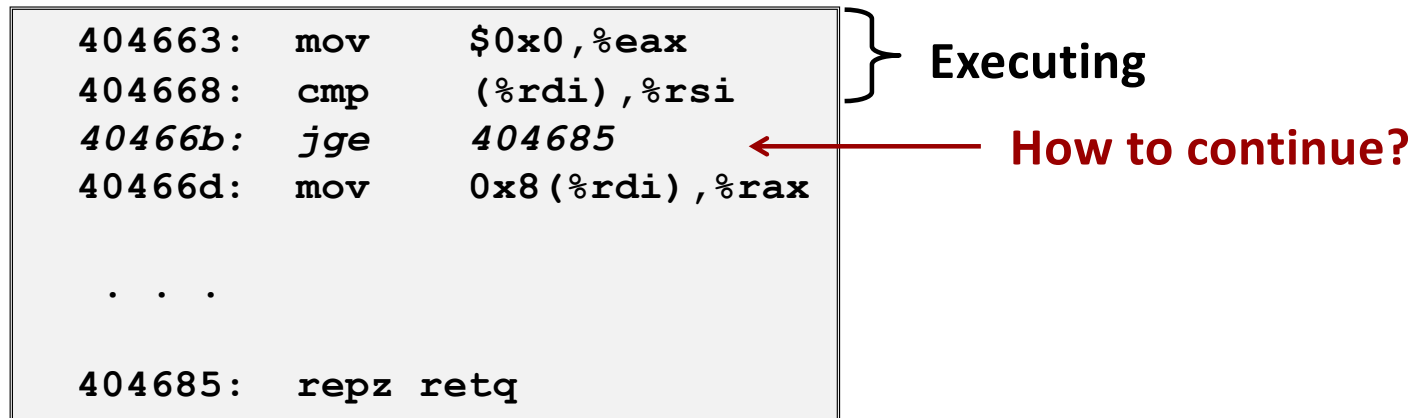
- Inner loop of the code with 20×20 unrolling

```
Updating of accumulator acc0 in 20 x 20 unrolling  
vmovsd 40(%rsp), %xmm0  
vmulsd (%rdx), %xmm0, %xmm0  
vmovsd %xmm0, 40(%rsp)
```

What About Branches?

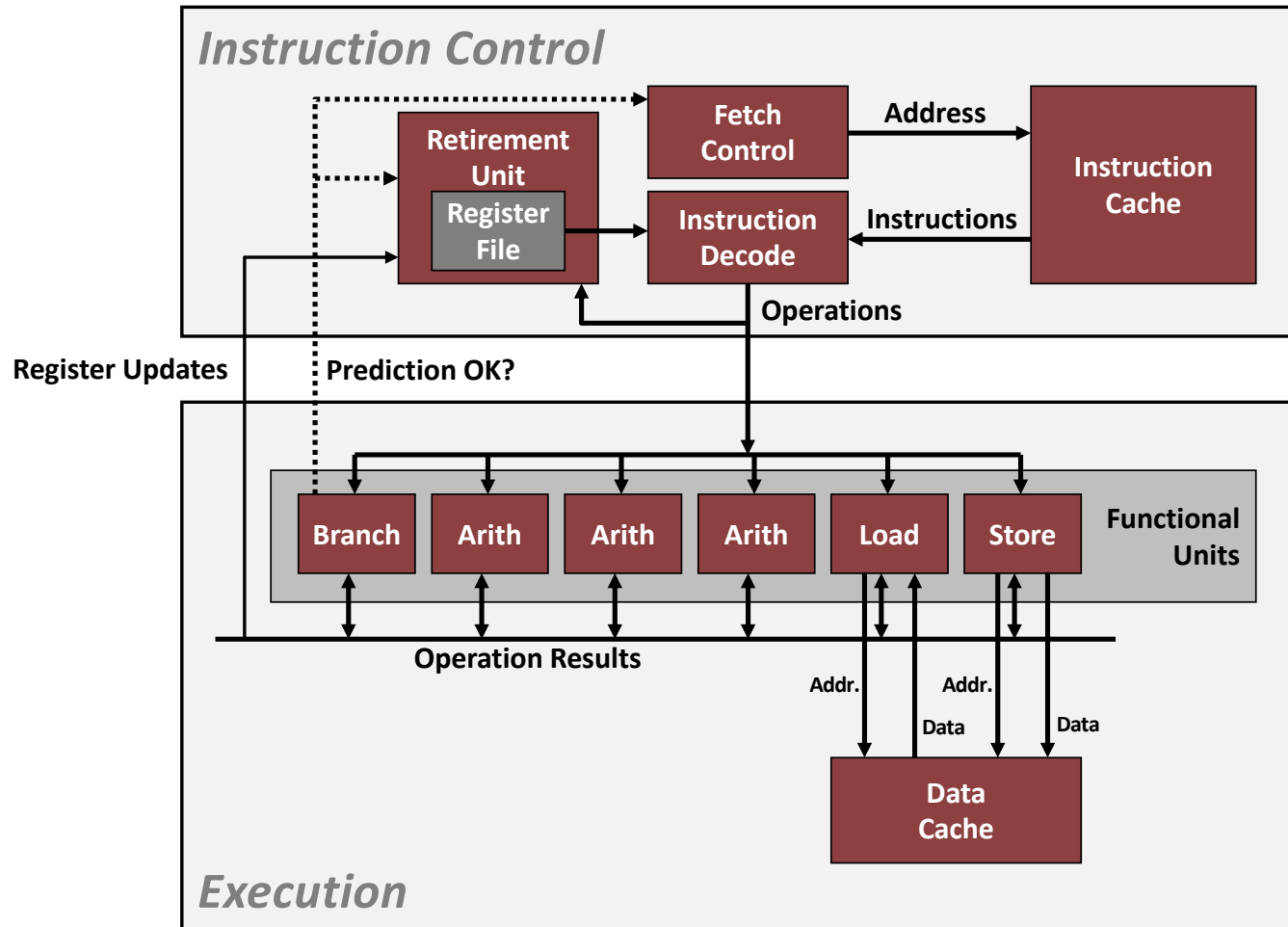
■ Challenge

- **Instruction Control Unit** must work well ahead of **Execution Unit** to generate enough operations to keep EU busy



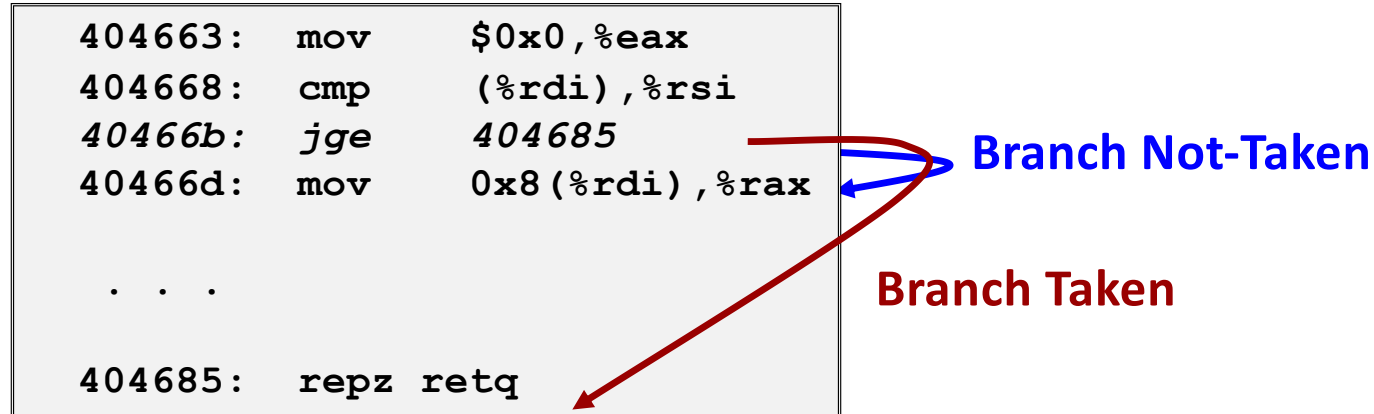
- When encounters conditional branch, cannot reliably determine where to continue fetching

Modern CPU Design



Branch Outcomes

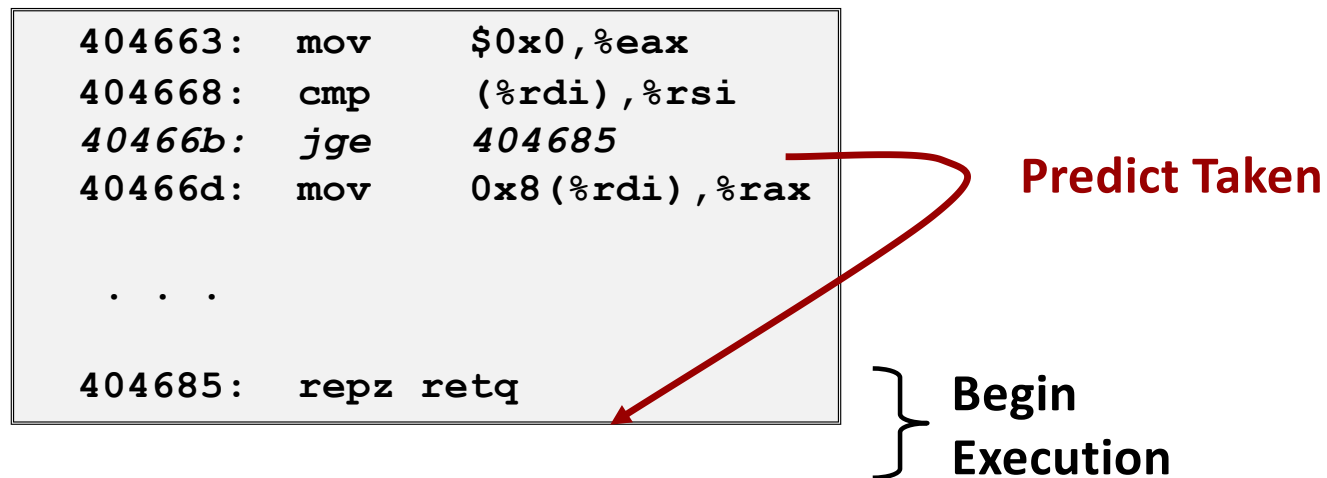
- When encounter conditional branch, cannot determine where to continue fetching
 - Branch Taken: Transfer control to branch target
 - Branch Not-Taken: Continue with next instruction in sequence
- Cannot resolve until outcome determined by branch/integer unit



Branch Prediction

■ Idea

- Guess which way branch will go
- Begin executing instructions at predicted position
 - But don't actually modify register or memory data



Branch Prediction Through Loop

```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029    i = 98
```

Assume
vector length = **100**

Predict Taken (OK)

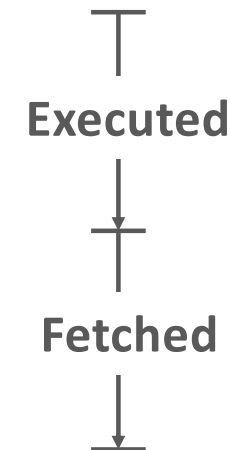
```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029    i = 99
```

Predict Taken
(Oops)

```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029    i = 100
```

Read
invalid
location

```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029    i = 101
```



Branch Misprediction Invalidation

```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029      i = 98
```

Assume
vector length = **100**

Predict Taken (OK)

```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029      i = 99
```

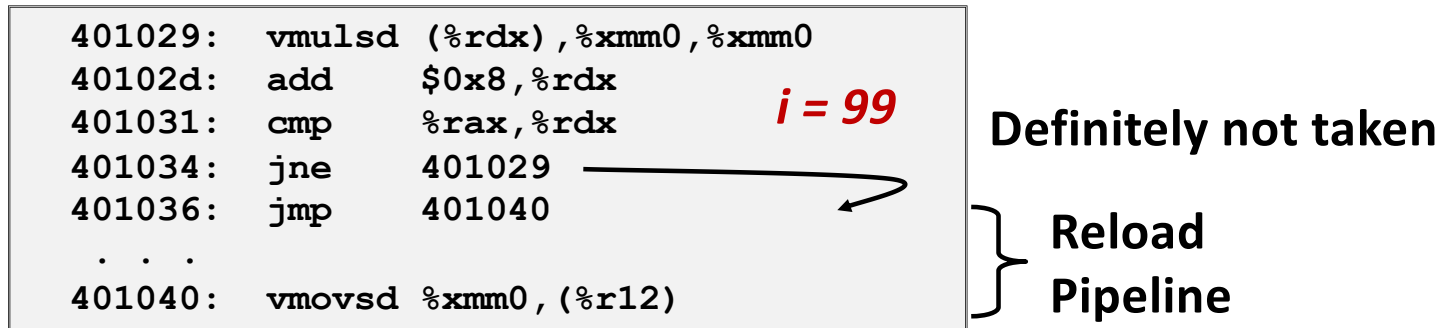
Predict Taken
(Oops)

```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029      i = 100
```

Invalidate

```
401029: vmulsd (%rdx), %xmm0, %xmm0
40102d: add    $0x8, %rdx
401031: cmp    %rax, %rdx
401034: jne    401029      i = 101
```

Branch Misprediction Recovery



■ Performance Cost

- Multiple clock cycles on modern processor
- Can be a major performance limiter

Do Not Be Overly Concerned about Predictable Branches

■ Combine2

- Move the call to `vec_length` out of the loop test

■ Combine3

- Take the function `get_vec_element` out of the inner loop of the function

Function	Page	Method	Integer		Floating point	
			+	*	+	*
<code>combine2</code>	509	Move <code>vec_length</code>	7.02	9.03	9.02	11.03
<code>combine3</code>	513	Direct data access	7.17	9.02	9.02	11.03

```

1  /* Move call to vec_length out of loop */
2  void combine2(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6
7      *dest = IDENT;
8      for (i = 0; i < length; i++) {
9          data_t val;
10         get_vec_element(v, i, &val);
11         *dest = *dest OP val;
12     }
13 }

```

Fig. 5.6 Improving the efficiency of the loop test.

```

1  /* Direct access to vector data */
2  void combine3(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      data_t *data = get_vec_start(v);
7
8      *dest = IDENT;
9      for (i = 0; i < length; i++) {
10         *dest = *dest OP data[i];
11     }
12 }

```

Fig. 5.7 Eliminating function calls within the loop.

```

int get_vec_element(vec_ptr v, long index, data_t *dest)
{
    if (index < 0 || index >= v->len)
        return 0;
    *dest = v->data[index];
    return 1;
}

```

Accessing Data Element with an Inline Function

```
1  /* Include bounds check in loop */
2  void combine4b(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      data_t acc = IDENT;
7
8      for (i = 0; i < length; i++) {
9          if (i >= 0 && i < v->len) {
10             acc = acc OP v->data[i];
11         }
12     }
13     *dest = acc;
14 }
```

Function	Page	Method	Integer		Floating point	
			+	*	+	*
combine4	515	No bounds checking	1.27	3.01	3.01	5.01
combine4b	515	Bounds checking	2.02	3.01	3.01	5.01

Write Code Suitable for Implementation with Conditional Moves

```
1  /* Rearrange two vectors so that for each i, b[i] >= a[i] */
2  void minmax1(long a[], long b[], long n) {
3      long i;
4      for (i = 0; i < n; i++) {
5          if (a[i] > b[i]) {
6              long t = a[i];
7              a[i] = b[i];
8              b[i] = t;
9          }
10     }
11 }
```

- CPE 13.5 for random data
- CPE 2.5–3.5 for predictable data, an indication of a misprediction penalty of around 20 cycles.

Write Code Suitable for Implementation with Conditional Moves

```
1  /* Rearrange two vectors so that for each i, b[i] >= a[i] */
2  void minmax2(long a[], long b[], long n) {
3      long i;
4      for (i = 0; i < n; i++) {
5          long min = a[i] < b[i] ? a[i] : b[i];
6          long max = a[i] < b[i] ? b[i] : a[i];
7          a[i] = min;
8          b[i] = max;
9      }
10 }
```

- **CPE of around 4.0 regardless of whether the data are arbitrary or predictable**

Getting High Performance

- **Good compiler and flags**
- **Don't do anything stupid**
 - Watch out for hidden algorithmic inefficiencies
 - Write compiler-friendly code
 - Watch out for optimization blockers:
procedure calls & memory references
 - Look carefully at innermost loops (where most work is done)
- **Tune code for machine**
 - Exploit instruction-level parallelism
 - Avoid unpredictable branches
 - Make code cache friendly